Complaint Pattern Analysis of Mumbai BMC Civic Complaints

Data Warehousing & Data Mining (CSI3010) — Lab Project Week 1

Team: Anshul Singh (24MID0291), Aarav Jha (24MID0285), Omkar Londhe (24MIC0135)

Faculty: Dr. Adrija Bhattacharya | Slot: L55+L56 | Date: 09-10-2026

Dataset Description

Property	Details
Dataset name	Mumbai BMC Civic Complaints
Source	Kaggle competition MCGM / BMC grievance system
Size	Train: 960,000 × 35 columns Test: 240,000 × 34 columns
Time period	2018-01-01 to 2024-12-30
Format	CSV files
Domain	Municipal governance / Smart Cities

Why this dataset?

Compare complaint categories and ward hotspots.

Look at seasonal resolution delays and contractor outcomes.

Audit result: zero missing values, zero duplicate complaint IDs.

Planned mining: classification, ward clustering, association rules, satisfaction prediction.

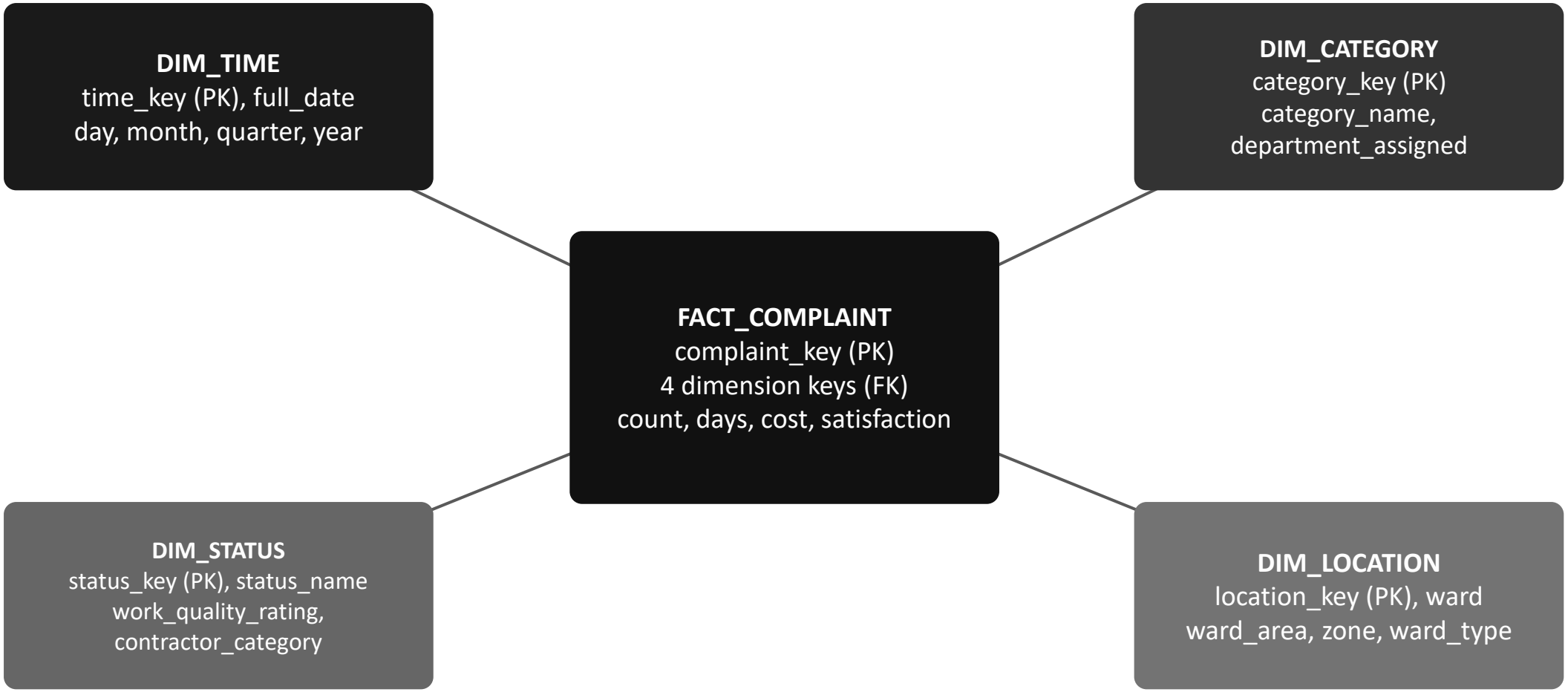
Attribute Summary: Keys & Dimensions

Attribute	Type	Description	Role in DW
complaint_id	String	Unique complaint identifier	Key
complaint_date	Date	Filing date	Dimension
year, month	Integer	Filing year and month	Dimension
is_monsoon_season	Integer	1 for June–September	Dimension
complaint_time_of_day	String	Time band of filing	Dimension
ward_code, ward_area	String	Ward code and locality	Dimension
zone, ward_type	String	Administrative zone and sector	Dimension
population_density	String	Population density class	Dimension
ward_slum_percentage	Decimal	Ward slum / chawl percentage	Dimension
complaint_category	String	Civic complaint category	Dimension
department_assigned	String	Responsible BMC department	Dimension
complaint_channel, severity	String	Filing channel and severity	Dimension
complaint_status	String	Final resolution status	Dimension
contractor_category, work_quality_rating	String	Contractor class and repair quality	Dimension
complainant_type, property_type	String	Submitter and property categories	Dimension

Attribute Summary: Measures & Flags

Attribute	Type	Description	Role in DW
resolution_days	Integer	Days taken to resolve	Measure
estimated_cost_inr	Decimal	Estimated cost in INR	Measure
num_reassignments	Integer	Times reassigned between departments	Measure
prior_complaints_count	Integer	Complainant's prior complaints	Measure
infrastructure_age_years	Integer	Age of affected infrastructure	Measure
months_since_last_maintained	Integer	Months since maintenance	Measure
citizen_satisfied	Integer	Satisfaction outcome (1 / 0)	Measure
has_photo_evidence, has_gps_location	Integer	Evidence and location flags	Fact flags
media_attention, politically_sensitive	Integer	Attention and sensitivity flags	Fact flags
repeat_complainant	Integer	Repeat complainant flag	Fact flag
site_inspected, defect_liability_claim	Integer	Inspection and liability flags	Fact flags
complaint_count (derived)	Integer	Constant 1 per complaint	Measure

Data Warehouse — Star Schema



Warehouse Inventory: All Dimensions & Fact Attributes

DIM_TIME

2,556 rows

KEYS
time_key (PK)
full_date (unique)

COLUMNS
day, month, month_name, quarter, year, day_of_week, is_monsoon_season

HIERARCHY
Day → Month → Quarter → Year

DIM_LOCATION

24 rows

KEYS
location_key (PK)
ward (unique)

COLUMNS
ward_area, zone, ward_type, population_density, ward_slum_percentage

HIERARCHY
Area → Ward → Zone

DIM_CATEGORY

13 rows

KEYS
category_key (PK)
category_name (unique)

COLUMNS
department_assigned

HIERARCHY
Complaint Category → Department

DIM_STATUS

100 rows

KEYS
status_key (PK)
status_name

COLUMNS
work_quality_rating, contractor_category

FACT_COMPLAINT

960,000 rows

One row per unique complaint

KEYS
complaint_key (PK)
complaint_id (unique)
FKs: time_key, location_key, category_key, status_key

MEASURES
complaint_count, resolution_days, estimated_cost_inr, num_reassignments, prior_complaints_count, infrastructure_age_years, months_since_last_maintained, citizen_satisfied

FLAGS AND CONTEXT
Channel, time of day, severity, complainant and property type, photo and GPS, media attention, political sensitivity, repeat complainant, inspection, defect liability

Creating the Warehouse: MySQL DDL (1/3)

```
CREATE DATABASE IF NOT EXISTS bmc_complaint_dw;
USE bmc_complaint_dw;

SET FOREIGN_KEY_CHECKS = 0;
DROP TABLE IF EXISTS fact_complaint;
DROP TABLE IF EXISTS dim_status;
DROP TABLE IF EXISTS dim_category;
DROP TABLE IF EXISTS dim_location;
DROP TABLE IF EXISTS dim_time;

CREATE TABLE dim_time (
    time_key INT PRIMARY KEY,
    full_date DATE NOT NULL UNIQUE,
    day INT NOT NULL, month INT NOT NULL,
    month_name VARCHAR(15) NOT NULL,
    quarter INT NOT NULL, year INT NOT NULL,
    day_of_week VARCHAR(15) NOT NULL,
    is_monsoon_season INT NOT NULL
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

CREATE TABLE dim_location (
    location_key INT PRIMARY KEY,
    ward VARCHAR(10) NOT NULL UNIQUE,
    ward_area VARCHAR(100) NOT NULL,
    zone VARCHAR(50) NOT NULL,
    ward_type VARCHAR(50) NOT NULL,
    population_density VARCHAR(50) NOT NULL,
    ward_slum_percentage DECIMAL(5,2) NOT NULL
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

Creating the Warehouse: MySQL DDL (2/3)

```
CREATE TABLE dim_category (
    category_key INT PRIMARY KEY,
    category_name VARCHAR(100) NOT NULL UNIQUE,
    department_assigned VARCHAR(100) NOT NULL
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

CREATE TABLE dim_status (
    status_key INT PRIMARY KEY,
    status_name VARCHAR(50) NOT NULL,
    work_quality_rating VARCHAR(50) NOT NULL,
    contractor_category VARCHAR(50) NOT NULL
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

-- Dimension hierarchies used in analysis:
-- Time: Day -> Month -> Quarter -> Year
-- Location: Area -> Ward -> Zone
-- Category: Complaint Category -> Department
-- Status groups status, work quality and contractor category.

-- Create fact_complaint only after all four dimensions exist.
```

Creating the Warehouse: MySQL DDL (3/3)

```
CREATE TABLE fact_complaint (
    complaint_key INT AUTO_INCREMENT PRIMARY KEY,
    complaint_id VARCHAR(50) NOT NULL UNIQUE,
    time_key INT NOT NULL, location_key INT NOT NULL,
    category_key INT NOT NULL, status_key INT NOT NULL,
    complaint_channel VARCHAR(50) NOT NULL,
    complaint_time_of_day VARCHAR(50) NOT NULL,
    severity VARCHAR(50) NOT NULL, complainant_type VARCHAR(50) NOT NULL,
    property_type VARCHAR(50) NOT NULL,
    complaint_count INT NOT NULL DEFAULT 1,
    resolution_days INT NOT NULL,
    estimated_cost_inr DECIMAL(12,2) NOT NULL,
    num_reassignments INT NOT NULL, prior_complaints_count INT NOT NULL,
    infrastructure_age_years INT NOT NULL,
    months_since_last_maintained INT NOT NULL,
    citizen_satisfied INT NOT NULL,
    has_photo_evidence INT NOT NULL, has_gps_location INT NOT NULL,
    media_attention INT NOT NULL, politically_sensitive INT NOT NULL,
    repeat_complainant INT NOT NULL, site_inspected INT NOT NULL,
    defect_liability_claim INT NOT NULL,
    CONSTRAINT fk_fact_time FOREIGN KEY (time_key) REFERENCES dim_time(time_key),
    CONSTRAINT fk_fact_location FOREIGN KEY (location_key) REFERENCES dim_location(location_key),
    CONSTRAINT fk_fact_category FOREIGN KEY (category_key) REFERENCES dim_category(category_key),
    CONSTRAINT fk_fact_status FOREIGN KEY (status_key) REFERENCES dim_status(status_key)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

CREATE INDEX idx_fact_time ON fact_complaint(time_key);
CREATE INDEX idx_fact_location ON fact_complaint(location_key);
CREATE INDEX idx_fact_category ON fact_complaint(category_key);
CREATE INDEX idx_fact_status ON fact_complaint(status_key);
SET FOREIGN_KEY_CHECKS = 1;
```

ETL Pipeline: Source-to-Warehouse Transformations

1 Extract

- Read dataset/bmc_train.csv with pandas.
- Input: 960,000 rows × 35 columns.
- Keep bmc_test.csv separate; citizen_satisfied is withheld there.

2 Clean & validate

- Work on a copy.
- Drop duplicate complaint_ids; standardize dates, strings and numeric types.
- Check for missing values and negative measures.

3 Engineer attributes

- Build the calendar fields for dim_time: day, month, month name, quarter, year and day of week.
- Add the monsoon flag and set complaint_count = 1.

4 Build dimensions

- Pull out the unique members for time, location, category and status.
- Assign surrogate keys and build lookup maps for the fact rows.

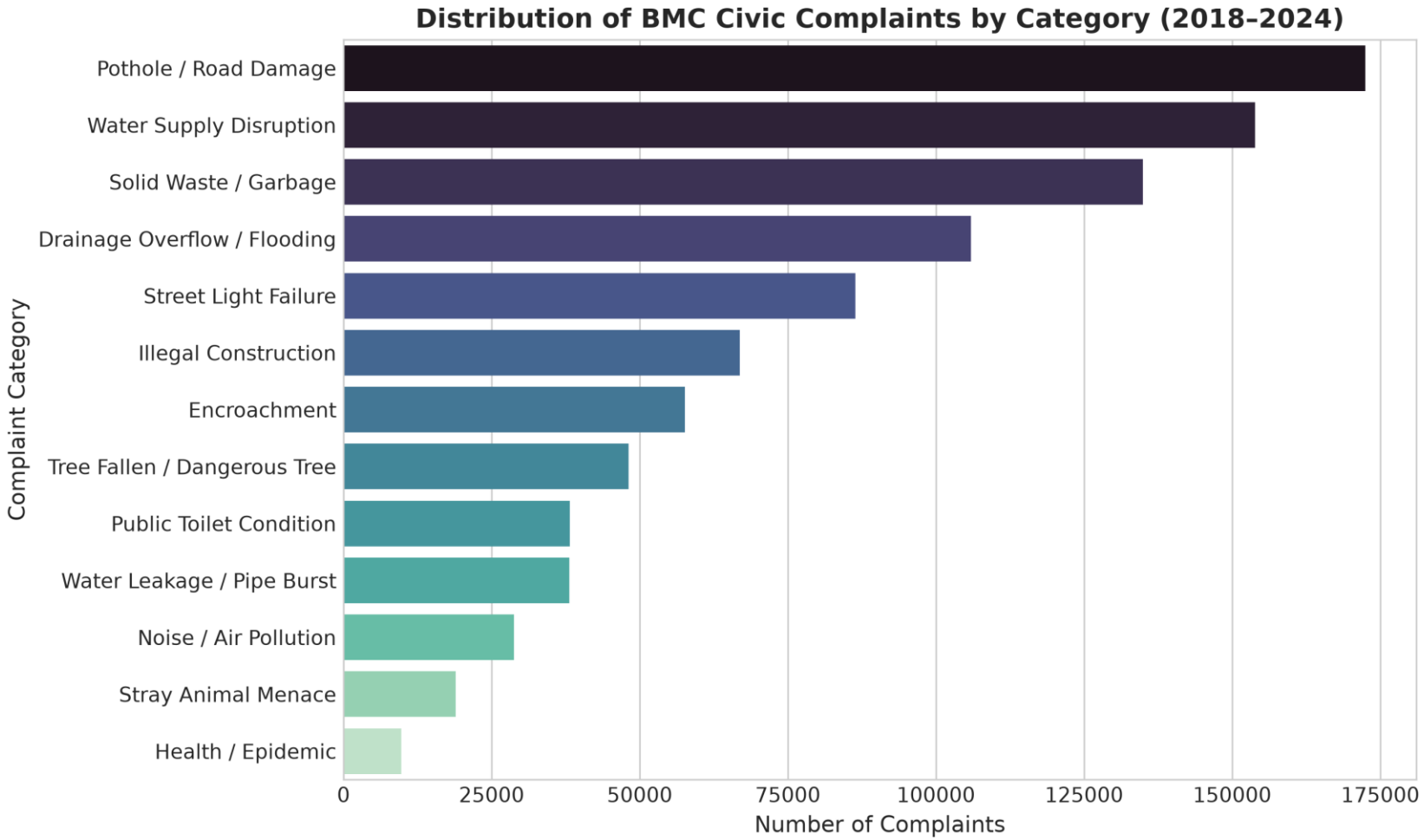
5 Build fact table

- One fact row per unique complaint_id.
- Map time_key, location_key, category_key and status_key.
- Keep measures and complaint flags.

6 Load & verify

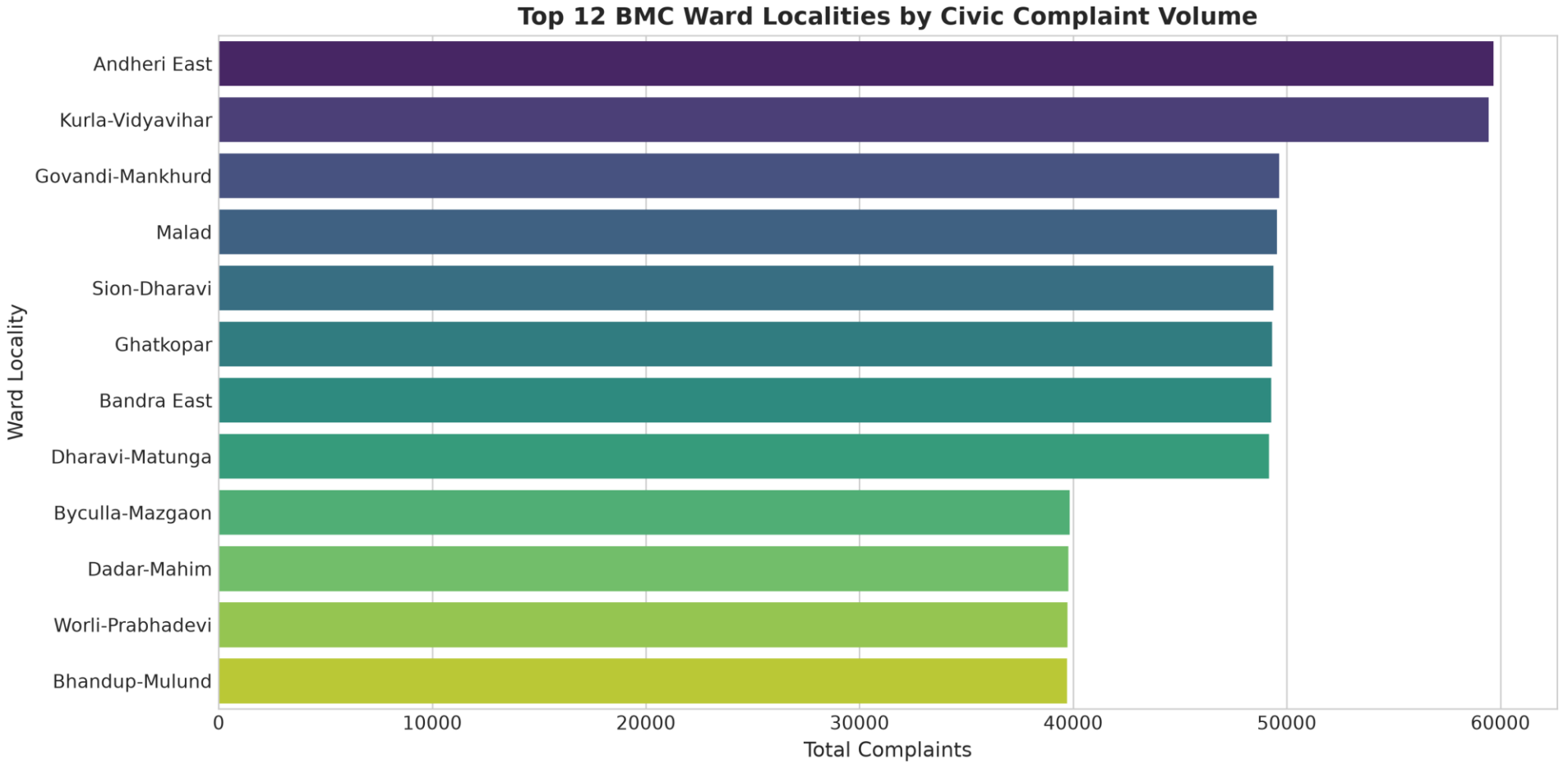
- Load dimensions first, then fact_complaint, using pandas / SQLAlchemy / MySQL.
- Check row counts, primary keys, NULLs and all four foreign-key joins.

Dataset: Complaint Categories



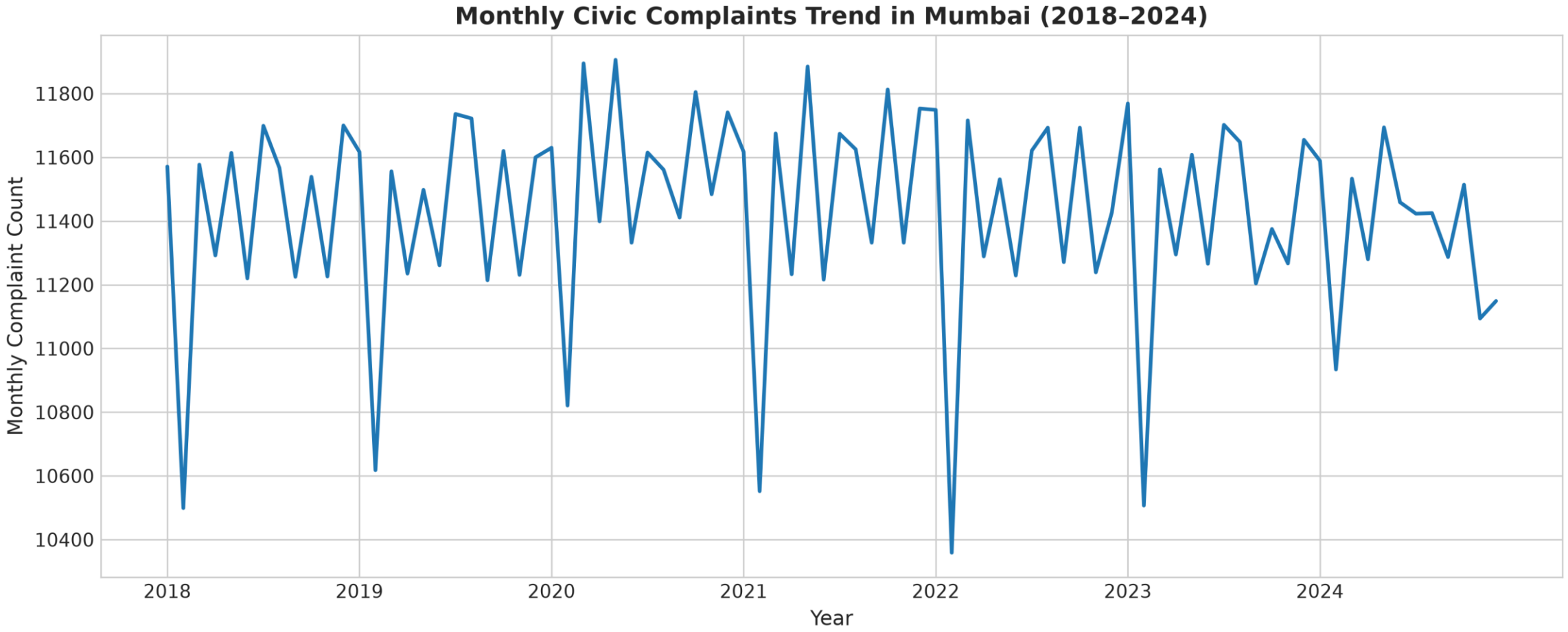
Pothole / Road Damage is the largest complaint category.

Dataset: Ward Hotspots



Andheri East and Kurla-Vidyavihar lead complaint volume.

Dataset: Monthly Complaint Trend



The chart shows monthly complaint counts from 2018 to 2024.

Warehouse Design & ETL Results

Grain: one row per unique complaint_id | Load dimensions before fact_complaint so the foreign keys resolve

1 EXTRACT

Read bmc_train.csv (960,000 × 35) with pandas. Keep bmc_test.csv separate because citizen_satisfied is withheld.

2 PROFILE & CLEAN

Check schema and data types, parse complaint_date, standardize string, date and measure formats, drop duplicate complaint_id values.

3 TRANSFORM

Add calendar fields (day, month, month_name, quarter, year, day_of_week) and the monsoon flag (June–September). Set complaint_count = 1.

4 BUILD DIMENSIONS

Create unique time, location, category and status records, with integer surrogate keys and lookup maps.

5 LOAD FACTS

Map the four FKs on each complaint row and keep the measures and binary flags. Load dimensions first, then fact_complaint.

Warehouse table	Loaded rows
fact_complaint	960,000
dim_time	2,556
dim_location	24
dim_category	13
dim_status	100

Notebook validation	Result
Orphan records across 4 FKs	0
Missing source values	0
Duplicate complaint IDs	0

Code: Python ETL & Loaded Results

```
df_raw = pd.read_csv(raw_train_path)
df_cleaned = df_raw.copy()
df_cleaned = (df_cleaned
 .drop_duplicates(subset=["complaint_id"])
 .reset_index(drop=True))

dim_time["time_key"] = dim_time.index + 1
fact_complaint["time_key"] = (
 df_cleaned["full_date"].map(time_lookup))

dim_time.to_sql("dim_time", engine,
 if_exists="append", index=False,
 chunksize=1000, method="multi")
fact_complaint.to_sql("fact_complaint", engine,
 if_exists="append", index=False,
 chunksize=20000, method="multi")
```

Excerpts cover extraction, key mapping and loading. Intermediate steps are left out.

Loaded warehouse join: sample rows

Key	Date	Ward	Days	Cost INR
1	2024-09-20	A	37	1,500
2	2024-01-02	P/N	12	38,600
3	2022-04-12	B	6	13,000
4	2024-02-28	M/E	1	2,400

960,000 fact records
0 orphan foreign keys

Dimensions load before facts.

OLAP Operations: Roll-up & Drill-down

Roll-up: ward to zone

```
SELECT l.zone,
 COUNT(f.complaint_key) AS total_complaints
FROM fact_complaint f
JOIN dim_location l
 ON f.location_key = l.location_key
GROUP BY l.zone
ORDER BY total_complaints DESC;
```

Zone	Complaints	Avg days
Western	355,984	13.47
City	346,769	13.8
Eastern	257,247	14.01

Western has the most complaints.

Drill-down: year to month

```
SELECT t.year, t.quarter, t.month,
 SUM(f.complaint_count) AS total_complaints
FROM fact_complaint f
JOIN dim_time t ON f.time_key = t.time_key
WHERE t.year = 2024
GROUP BY t.year, t.quarter, t.month, t.month_name
ORDER BY t.quarter, t.month;
```

2024 month	Complaints	Avg days
May	11,694	12.23
June	11,459	16.55
July	11,423	16.68
September	11,287	16.94

September's average resolution time is 16.94 days.

OLAP Operations: Slice & Dice

Slice: one category

```
WHERE c.category_name =
 'Pothole / Road Damage'
GROUP BY l.ward, l.ward_area, l.zone
ORDER BY pothole_complaints DESC;
```

Dice: category, zone and year

```
WHERE c.category_name IN (
 'Pothole / Road Damage',
 'Water Supply Disruption',
 'Drainage Overflow / Flooding')
 AND l.zone IN ('Western', 'Eastern')
 AND t.year = 2024
```

Filter and grouping excerpts from the full queries

Ward	Potholes	Avg days
K/E	10,668	15.05
L	10,596	18.17
H/E	8,901	15.17

K/E leads pothole complaints.

2024 zone / category	Count	Days
Eastern / Pothole	6,450	15.64
Western / Pothole	9,107	15.15
Eastern / Water supply	5,903	9.33
Western / Water supply	8,052	9.18

Eastern potholes take 15.64 days to resolve.

OLAP Operations: Pivot

Ward rows with complaint categories as columns

```
SELECT l.ward, l.ward_area,
 SUM(CASE WHEN c.category_name =
 'Pothole / Road Damage'
 THEN f.complaint_count ELSE 0 END)
 AS pothole_roads,
 SUM(CASE WHEN c.category_name =
 'Water Supply Disruption'
 THEN f.complaint_count ELSE 0 END)
 AS water_supply,
 SUM(f.complaint_count)
 AS total_ward_complaints
FROM fact_complaint f
JOIN dim_location l
 ON f.location_key = l.location_key
JOIN dim_category c
 ON f.category_key = c.category_key
GROUP BY l.ward, l.ward_area
ORDER BY total_ward_complaints DESC;
```

Result table (selected rows / columns)

Ward	Potholes	Water	Total
K/E	10,668	9,540	59,673
L	10,596	9,620	59,459
M/E	8,894	7,998	49,648
P/N	8,875	7,940	49,548

K/E has the most complaints overall: 59,673.